



NETLINE

Guia de Estudos para Estagiários

Fundamentos de Programação até Desenvolvimento Web

Introdução

Bem-vindo à Netline! Este guia foi preparado especialmente para estagiários que estão a dar os primeiros passos no desenvolvimento de software. O percurso está organizado em fases progressivas — cada fase constrói sobre a anterior — e cobre as tecnologias reais que usamos no dia a dia da empresa.

Não te preocupes se alguns conceitos parecerem difíceis ao início. A programação aprende-se com prática e tempo. Lê com calma, experimenta os exemplos, e não hesites em perguntar à equipa.

Como usar este guia

Segue as fases na ordem apresentada. Para cada tema, lê a definição introdutória, consulta os materiais recomendados e tenta fazer pequenos exercícios práticos. Não é necessário memorizar — o importante é compreender os conceitos e saber onde procurar quando precisares.

Mapa do Percurso de Aprendizagem

O guia está dividido em 6 fases, com uma duração estimada total de 3 a 5 meses de estudo consistente:

#	Fase	Tema Principal	Duração Estimada
1	Fundamentos	Lógica, algoritmos e pensamento computacional	3–4 semanas
2	Git & Terminal	Controlo de versões e linha de comandos	1–2 semanas
3	HTML, CSS & JS	Bases do desenvolvimento web frontend	4–6 semanas
4	TypeScript & React	Frontend moderno com tipagem estática	4–6 semanas
5	Backend	Node.js, Java, Spring Boot e PostgreSQL	6–8 semanas
6	REST & Boas Práticas	Arquitetura REST, Docker e CI/CD	3–4 semanas

Antes de escrever qualquer linha de código, é essencial compreender como os computadores "pensam" e como estruturamos soluções para problemas. Esta fase foca-se em conceitos universais que se aplicam a qualquer linguagem.

1.1 O que é Programação?

Programar é dar instruções precisas a um computador para que ele execute uma tarefa. Um computador faz exatamente o que lhe dizes — nem mais, nem menos. A arte da programação está em decompor problemas complexos em passos simples e ordenados.

Conceito: Algoritmo

Um algoritmo é uma sequência finita e ordenada de passos bem definidos para resolver um problema. Exemplo: a receita de cozinha de um prato é um algoritmo — tem ingredientes (dados de entrada), passos (instruções) e um resultado (saída).

Conceito: Programa vs. Algoritmo

Um algoritmo é a ideia/lógica. Um programa é o algoritmo escrito numa linguagem que o computador consegue executar (JavaScript, Python, Java, etc.).

1.2 Lógica e Estruturas Fundamentais

Toda a programação é construída sobre três estruturas básicas de controlo de fluxo:

- **Sequência** — Sequência: instruções executadas uma após a outra, em ordem.
- **Condição** — Condição (if/else): executa um bloco de código apenas se uma condição for verdadeira.
- **Repetição** — Repetição (loop/ciclo): repete um bloco de código enquanto uma condição for verdadeira.

Conceito: Variável

Uma variável é um espaço na memória do computador com um nome, onde guardamos um valor que pode mudar. Exemplo: idade = 22. O nome 'idade' aponta para o valor 22 na memória.

Conceito: Função

Uma função é um bloco de código nomeado e reutilizável que executa uma tarefa específica. Podes chamá-la várias vezes sem repetir o código. Funções recebem dados (parâmetros) e podem devolver um resultado (retorno).

1.3 Tipos de Dados

Todo o valor num programa tem um tipo. Os tipos básicos (primitivos) são:

- String: texto (ex: "Olá, mundo!")
- Number / Integer / Float: números inteiros ou decimais (ex: 42, 3.14)
- Boolean: verdadeiro ou falso (true / false)
- Null / Undefined: ausência de valor

Conceito: Array (Lista)

Um array é uma coleção ordenada de valores do mesmo tipo, acessíveis por índice numérico. Exemplo: nomes = ["Ana", "Bruno", "Carlos"]. O primeiro elemento tem índice 0.

Conceito: Objeto / Dicionário

Um objeto é uma coleção de pares chave-valor. Exemplo: { nome: "Ana", idade: 25 }. Permite representar entidades com múltiplas propriedades.

1.4 Materiais de Estudo — Fase 1

Leitura Recomendada

- [CS50 — Introduction to Computer Science \(Harvard, gratuito\)](#)
- [Curso em Vídeo — Lógica de Programação \(Português\)](#)
- [freeCodeCamp — JavaScript Algorithms and Data Structures](#)

Prática

- [Exercism.io — exercícios práticos em várias linguagens](#)
- [CodeWars — desafios de lógica progressivos](#)

Na Netline usamos Git diariamente para gerir o código de todos os projetos. Aprender Git cedo vai poupar-te muito tempo e frustração. O terminal é a ferramenta principal de trabalho de qualquer programador.

2.1 O Terminal

O terminal (também chamado de shell ou linha de comandos) é uma interface de texto para interagir com o sistema operativo. Em vez de clicar em ícones, escreves comandos. É mais rápido e poderoso para tarefas de desenvolvimento.

Comandos Essenciais (Linux/macOS)

pwd — mostra o diretório atual *cd pasta/* — entra numa pasta — lista ficheiros e pastas
mkdir nome — cria uma pasta *touch ficheiro.txt* — cria um ficheiro *cat ficheiro.txt* — mostra o conteúdo do ficheiro — remove um ficheiro

2.2 O que é Git?

Git é um sistema de controle de versões distribuído. Permite guardar o histórico de alterações do código, trabalhar em equipa sem conflitos, e voltar a versões anteriores quando algo corre mal.

Conceito: Repositório

Um repositório (repo) é a pasta do projeto gerida pelo Git. Contém todo o código e o histórico completo de todas as alterações feitas.

Conceito: Commit

Um commit é um "ponto de save" do teu código. Cada commit tem uma mensagem descritiva (ex: 'Adiciona formulário de login') e fica registado no histórico para sempre.

Conceito: Branch (Ramo)

Uma branch é uma linha de desenvolvimento paralela. Podes criar uma branch para desenvolver uma nova funcionalidade sem afetar o código principal (branch main). Quando terminares, fazes merge para juntar as alterações.

Conceito: GitHub / GitLab

GitHub e GitLab são plataformas online para hospedar repositórios Git. Permitem colaboração em equipa, revisão de código (Pull Requests) e integração com ferramentas de CI/CD. Na Netline usamos GitHub.

2.3 Comandos Git Essenciais

- `git init` — inicializa um repositório na pasta atual
- `git clone <url>` — copia um repositório remoto para a tua máquina
- `git status` — mostra o estado atual das alterações
- `git add .` — prepara todas as alterações para commit
- `git commit -m "mensagem"` — guarda as alterações com uma mensagem
- `git push` — envia os commits para o repositório remoto
- `git pull` — traz as alterações remotas para a tua máquina
- `git checkout -b nova-branch` — cria e entra numa nova branch

2.4 Materiais de Estudo — Fase 2

Git

- [Pro Git Book \(gratuito, disponível em Português\)](#)
- [GitHub Skills — aprende Git interativamente](#)
- [Learn Git Branching — visualização interativa de branches](#)

Terminal

- [The Linux Command Line \(livro gratuito online\)](#)

Estas três tecnologias são a base de toda a web. Qualquer página ou aplicação web que abrires no browser é construída com HTML, CSS e JavaScript. É essencial dominá-las antes de avançar para frameworks modernos.

3.1 HTML — Estrutura

O que é HTML?

HTML (HyperText Markup Language) é a linguagem de marcação que define a estrutura e o conteúdo de uma página web. Não é uma linguagem de programação — não tem lógica. Define o que está na página: títulos, parágrafos, imagens, links, formulários.

HTML é escrito em tags — elementos entre `< >`. Cada elemento tem uma tag de abertura e uma de fecho:

- `<h1>` a `<h6>` — títulos de diferentes níveis
- `<p>` — parágrafo de texto
- `<a href="url">` — hiperligação
- `` — imagem
- `<div>` e `<span>` — contêineres genéricos
- `<form>`, `<input>`, `<button>` — formulários e interação

3.2 CSS — Aparência

O que é CSS?

CSS (Cascading Style Sheets) define a aparência visual dos elementos HTML. Controla cores, fontes, tamanhos, espaçamentos, posicionamento e animações. Define o COMO parece a página.

Conceito: Box Model

Todo o elemento HTML é tratado como uma caixa rectangular com: conteúdo (content), espaçamento interno (padding), borda (border) e margem externa (margin). Compreender o box model é essencial para fazer layouts.

Conceito: Flexbox & Grid

Flexbox e CSS Grid são sistemas modernos de layout. Flexbox organiza elementos numa dimensão (linha ou coluna). Grid organiza em duas dimensões (linhas e colunas). São as ferramentas padrão para construir layouts responsivos hoje em dia.

3.3 JavaScript — Comportamento

O que é JavaScript?

JavaScript (JS) é a linguagem de programação da web. É executada diretamente no browser e permite tornar as páginas dinâmicas e interativas: responder a cliques, validar formulários, carregar dados sem recarregar a página, etc.

Conceito: DOM

DOM (Document Object Model) é a representação do HTML em memória como uma árvore de objetos. O JavaScript pode ler e modificar o DOM para alterar dinamicamente o conteúdo e a aparência da página.

Conceito: Eventos

Eventos são ações do utilizador (clique, tecla premida, scroll) ou do sistema (página carregada). Em JavaScript, podes "ouvir" eventos com `addEventListener` e executar código em resposta.

Conceito: Promises & Async/Await

Operações demoradas (como buscar dados de um servidor) são assíncronas — o código não espera que terminem para continuar. Promises e `async/await` são formas modernas de lidar com operações assíncronas de forma legível.

3.4 Materiais de Estudo — Fase 3

HTML & CSS

- [MDN Web Docs — HTML \(referência completa, gratuita\)](#)
- [MDN Web Docs — CSS](#)
- [CSS-Tricks — Guia completo de Flexbox](#)
- [freeCodeCamp — Responsive Web Design Certification](#)

JavaScript

- [javascript.info — O tutorial mais completo de JS \(tem tradução PT\)](#)
- [Eloquent JavaScript \(livro gratuito online\)](#)
- [freeCodeCamp — JavaScript Algorithms and Data Structures](#)

Na Netline, o nosso frontend é construído com React e TypeScript. Depois de dominares JavaScript, TypeScript será uma evolução natural, e React irá transformar a forma como pensas sobre interfaces.

4.1 TypeScript

O que é TypeScript?

TypeScript é JavaScript com tipagem estática. Isso significa que defines explicitamente o tipo de cada variável e parâmetro de função. O compilador TypeScript deteta erros antes de executares o código, tornando projetos grandes muito mais seguros e fáceis de manter.

Conceito: Tipagem Estática vs. Dinâmica

Em JavaScript (dinâmica), uma variável pode conter qualquer tipo de valor em qualquer momento. Em TypeScript (estática), defines o tipo na declaração e o compilador garante que nunca mudarás para um tipo incompatível. Isto evita uma classe enorme de bugs.

Conceito: Interface & Type

Em TypeScript, interface e type permitem descrever a forma de um objeto. Exemplo: `interface User { nome: string; idade: number }`. Qualquer objeto que use este tipo tem de ter exatamente esses campos com esses tipos.

Conceito: Generics

Generics permitem criar funções e classes que funcionam com qualquer tipo, mantendo a segurança de tipos. Exemplo: `Array<string>` é um array que só pode conter strings. `Array<number>` só pode conter números.

4.2 React

O que é React?

React é uma biblioteca JavaScript para construir interfaces de utilizador. Em vez de manipular o DOM diretamente, defines Componentes — pequenas peças reutilizáveis de UI — e o React trata de atualizar eficientemente o que aparece no ecrã quando os dados mudam.

Conceito: Componente

Um componente React é uma função que recebe dados (props) e devolve HTML (JSX). É a unidade básica de construção de interfaces em React. Uma página inteira é construída como uma árvore de componentes.

Conceito: Props

Props (propriedades) são os dados passados de um componente pai para um componente filho. São só de leitura — o filho não pode modificar as props que recebe. É assim que os componentes comunicam entre si.

Conceito: State (Estado)

State é o estado interno de um componente — dados que podem mudar ao longo do tempo e que, quando mudam, fazem o componente re-renderizar. O hook `useState` é a forma principal de gerir estado em React moderno.

Conceito: Hooks

Hooks são funções especiais do React (começam por 'use') que permitem usar funcionalidades do React dentro de componentes funcionais. Os mais importantes: `useState` (estado), `useEffect` (efeitos colaterais), `useContext` (contexto global).

Conceito: shadcn/ui

Na Netline usamos `shadcn/ui` — uma coleção de componentes React reutilizáveis e acessíveis construídos sobre `Radix UI` e `Tailwind CSS`. Oferece botões, formulários, modais, tabelas e muito mais, prontos a personalizar.

4.3 Next.js

O que é Next.js?

Next.js é uma framework construída sobre React que adiciona funcionalidades essenciais para produção: routing baseado em ficheiros, Server-Side Rendering (SSR), Static Site Generation (SSG), API Routes integradas e otimizações automáticas de performance. É o que usamos nos projetos de frontend da Netline (ex: SIGECAD/UJC).

4.4 Materiais de Estudo — Fase 4

TypeScript

- [TypeScript Handbook \(documentação oficial\)](#)
- [Total TypeScript — Beginners Tutorial \(gratuito\)](#)

React

- [React — Documentação Oficial \(nova, com exemplos interativos\)](#)
- [Scrimba — Learn React for Free](#)

Next.js

- [Next.js — Learn \(tutorial oficial interativo\)](#)
- [shadcn/ui — Documentação de componentes](#)

O backend é a parte do sistema que corre no servidor — recebe pedidos, processa lógica de negócio, comunica com a base de dados e devolve respostas. Na Netline usamos duas stacks de backend complementares: Node.js com TypeScript para serviços mais ágeis, e Java com Spring Boot para sistemas empresariais mais robustos como o FICA. PostgreSQL é a base de dados comum a ambas.

Duas Stacks, Um Objetivo

Não te preocupes em dominar ambas ao mesmo tempo. Começa pelos conceitos comuns (HTTP, bases de dados, APIs REST) e depois aprofunda cada tecnologia. Os princípios são os mesmos — muda a linguagem e as ferramentas.

5.1 Bases de Dados & SQL

Este tema é transversal a ambas as stacks. Antes de aprender Node.js ou Java, é essencial perceber como funcionam as bases de dados relacionais.

O que é uma Base de Dados Relacional?

Uma base de dados relacional organiza os dados em tabelas com linhas e colunas, relacionadas entre si. Cada tabela representa uma entidade (ex: users, products). As tabelas ligam-se através de chaves estrangeiras (foreign keys). Usamos PostgreSQL, um dos sistemas mais robustos e populares.

Conceito: SQL

SQL (Structured Query Language) é a linguagem universal para comunicar com bases de dados relacionais. Principais operações: SELECT (ler), INSERT (inserir), UPDATE (atualizar), DELETE (apagar). É usada tanto em projetos Node.js como em Java — é uma competência fundamental.

Conceito: ORM

ORM (Object-Relational Mapping) é uma camada que permite interagir com a base de dados usando objetos da linguagem de programação, sem escrever SQL manualmente. No ecossistema Node.js usamos Prisma; no ecossistema Java/Spring Boot usamos JPA com Hibernate.

5.2 Stack Node.js — Express & Prisma

Usada em projetos como o CNEAD. É a stack mais rápida de montar e ideal para APIs REST com TypeScript.

O que é Node.js?

Node.js é um ambiente de execução JavaScript fora do browser — permite correr JavaScript no servidor. É assíncrono e orientado a eventos, muito eficiente para APIs que lidam com muitos pedidos simultâneos.

Conceito: npm (Node Package Manager)

npm é o gestor de pacotes do Node.js. Permite instalar e gerir as bibliotecas do projeto. O ficheiro package.json lista todas as dependências e scripts. yarn e pnpm são alternativas populares ao npm.

O que é Express.js?

Express é a framework web mais popular para Node.js. Permite criar um servidor HTTP e definir rotas — mapeamentos entre URLs e funções que processam os pedidos. É minimalista e flexível.

Conceito: Middleware (Express)

Middleware é uma função executada entre a receção de um pedido HTTP e o envio da resposta. Usada para autenticação, validação, logging e tratamento de erros. No Express, os middlewares encadeiam-se com next().

Conceito: Prisma ORM

Prisma é um ORM moderno para Node.js e TypeScript. Define o schema da base de dados num ficheiro .prisma, gera um cliente TypeScript tipado e faz migrações automaticamente. Garante que os tipos dos dados da base de dados estão sempre sincronizados com o código TypeScript.

5.3 Stack Java — Spring Boot & JPA

Usada em projetos como o FICA CORE. Java é uma das linguagens mais usadas no mundo empresarial, com ecossistema maduro, alto desempenho e tipagem forte. Spring Boot é a framework de referência para APIs Java em produção.

O que é Java?

Java é uma linguagem de programação orientada a objetos, compilada, fortemente tipada e independente de plataforma (corre na JVM — Java Virtual Machine). É amplamente usada em sistemas empresariais, backends de grande escala e aplicações Android. É mais verbosa que TypeScript mas oferece robustez e ferramentas de refactoring excelentes.

Conceito: Programação Orientada a Objetos (POO)

POO é um paradigma que organiza o código em Classes e Objetos. Uma Classe é um molde (ex: class User); um Objeto é uma instância desse molde (ex: user1 = new User()). Os

quatro pilares são: Encapsulamento (esconder detalhes internos), Herança (reutilizar comportamento de outra classe), Polimorfismo (um método com múltiplos comportamentos) e Abstração (expor só o essencial).

Conceito: Maven & Gradle

Maven e Gradle são ferramentas de build e gestão de dependências para Java — o equivalente ao npm no ecossistema Node.js. O ficheiro pom.xml (Maven) ou build.gradle (Gradle) declara as dependências do projeto, como spring-boot-starter-web ou spring-data-jpa.

O que é Spring Boot?

Spring Boot é a framework Java mais popular para construir APIs REST e aplicações empresariais. Elimina grande parte da configuração manual do Spring tradicional — com convenções inteligentes e auto-configuração, tens uma API funcional com poucas linhas de código. Usado no FICA CORE para relatórios académicos e gestão de desempenho.

Conceito: Anotações Spring (@)

Spring Boot usa anotações Java para configurar o comportamento das classes: @RestController — marca a classe como controlador de uma API REST @GetMapping / @PostMapping — mapeia um método a um endpoint HTTP @Service — marca uma classe como serviço com lógica de negócio @Repository — marca uma classe como repositório de dados @Autowired — injeta automaticamente uma dependência

Conceito: JPA & Hibernate

JPA (Java Persistence API) é o standard Java para ORM. Hibernate é a implementação mais popular de JPA. Com Spring Data JPA, defines uma interface que estende JpaRepository e o Spring gera automaticamente os métodos de CRUD (save, findById, findAll, delete, etc.) — sem escrever SQL manualmente na maioria dos casos.

Conceito: Injeção de Dependências

Um dos conceitos centrais do Spring é a Inversão de Controlo (IoC) — em vez de criares objetos manualmente com new, declarar as dependências e o Spring cria e injeta-as automaticamente. Isto torna o código mais modular, testável e fácil de manter.

Conceito: Camadas de uma Aplicação Spring Boot

Uma aplicação Spring Boot típica organiza-se em camadas: Controller — recebe os pedidos HTTP e devolve respostas Service — contém a lógica de negócio Repository — comunica com a base de dados Model/Entity — representa as tabelas da base de dados como classes Java Esta separação facilita testes, manutenção e evolução do código.

5.4 Conceitos Comuns ao Backend

Conceito: Variáveis de Ambiente (.env / application.properties)

Configurações sensíveis — palavras-passe de bases de dados, chaves de API, portas — nunca devem estar no código fonte. Em Node.js usamos ficheiros .env com dotenv. Em Spring Boot usamos application.properties ou application.yml, com suporte a perfis (dev, prod) para diferentes ambientes.

Conceito: Tratamento de Erros

Um bom backend devolve erros informativos e consistentes. Em Express, usamos middlewares de erro globais. Em Spring Boot, usamos @ControllerAdvice com @ExceptionHandler para capturar exceções e devolver respostas HTTP padronizadas (ex: { erro: 'Utilizador não encontrado', status: 404 }).

5.5 Materiais de Estudo — Fase 5

Bases de Dados & SQL

- [PostgreSQL Tutorial \(completo e gratuito\)](#)
- [PlanetScale — Introduction to Databases for Developers](#)

Node.js & Express & Prisma

- [Node.js — Documentação Oficial](#)
- [The Odin Project — NodeJS Path \(gratuito, projeto-based\)](#)
- [Express.js — Guia Oficial](#)
- [Prisma — Getting Started \(documentação oficial\)](#)

Java & Spring Boot

- [Java — Curso Completo para Iniciantes \(Loiane Groner, gratuito, PT\)](#)
- [Spring Boot — Tutorial Oficial \(construir uma API do zero\)](#)
- [Baeldung — Guias práticos de Spring Boot \(referência\)](#)
- [Spring Data JPA — Documentação Oficial](#)
- [Amigoscode — Spring Boot para Iniciantes \(YouTube, gratuito\)](#)

Esta fase junta os conceitos anteriores e introduz as práticas profissionais que usamos em produção na Netline. REST define como as APIs devem ser desenhadas. Docker garante que o código corre igualmente em qualquer máquina. CI/CD automatiza a entrega de código.

6.1 Arquitetura REST

O que é REST?

REST (Representational State Transfer) é um estilo arquitetural para construir APIs web. Define um conjunto de princípios sobre como o servidor deve expor os seus recursos através do protocolo HTTP de forma simples, consistente e escalável.

Conceito: Recurso e URL

Em REST, tudo é um recurso — utilizadores, produtos, pedidos. Cada recurso tem uma URL única. Exemplo: /api/users representa todos os utilizadores; /api/users/42 representa o utilizador com id 42. As URLs devem ser substantivos, nunca verbos.

Conceito: Métodos HTTP

REST usa os métodos HTTP para indicar a ação sobre um recurso: GET /users → listar utilizadores GET /users/42 → obter utilizador 42 POST /users → criar novo utilizador PUT /users/42 → atualizar utilizador 42 (substituição total) PATCH /users/42 → atualizar campos específicos DELETE /users/42 → apagar utilizador 42

Conceito: Códigos de Status HTTP

Cada resposta HTTP tem um código de status que indica o resultado: 2xx — Sucesso (200 OK, 201 Created, 204 No Content) 4xx — Erro do cliente (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found) 5xx — Erro do servidor (500 Internal Server Error) Usar os códigos corretos é essencial para que o cliente saiba o que aconteceu.

Conceito: JSON

JSON (JavaScript Object Notation) é o formato de dados padrão das APIs REST. É texto estruturado como objetos JavaScript — fácil de ler por humanos e de processar por máquinas. Exemplo: { "id": 1, "nome": "Ana", "email": "ana@example.com" }

Conceito: Stateless (Sem Estado)

Um princípio fundamental do REST é que cada pedido ao servidor deve ser completamente independente — o servidor não guarda contexto entre pedidos. Toda a informação necessária (autenticação, dados) deve estar no próprio pedido. Isto torna as APIs mais escaláveis.

6.2 Autenticação em APIs

Conceito: JWT (JSON Web Token)

JWT é um padrão para transmitir informação de identidade de forma segura entre cliente e servidor. Após o login, o servidor gera um token assinado que o cliente envia em cada pedido subsequente no header `Authorization: Bearer <token>`. O servidor verifica a assinatura sem precisar consultar uma sessão.

6.3 Docker

O que é Docker?

Docker é uma plataforma de containerização. Um container é um pacote isolado que contém a aplicação e todas as suas dependências (Node.js, JDK, bibliotecas, variáveis de ambiente) — garante que o código corre exatamente da mesma forma em qualquer máquina, seja a tua, de um colega, ou no servidor de produção.

Conceito: Dockerfile

Um Dockerfile é um ficheiro de texto com instruções para construir uma imagem Docker. Define a imagem base (ex: `node:20` para Node.js, ou `eclipse-temurin:21` para Java), copia o código, instala dependências e define o comando de arranque da aplicação.

Conceito: Docker Compose

Docker Compose permite definir e executar aplicações com múltiplos serviços (ex: API + base de dados + Nginx) num único ficheiro `docker-compose.yml`. Com um único comando (`docker compose up`) tens todo o ambiente a correr.

6.4 CI/CD com GitHub Actions

Conceito: CI/CD

CI (Continuous Integration) é a prática de integrar e testar automaticamente o código a cada push. CD (Continuous Deployment) vai além e faz o deploy automático para produção se os testes passarem. Na Netline usamos GitHub Actions para automatizar testes, builds Docker e deploys.

6.5 Materiais de Estudo — Fase 6

REST & HTTP

- [RESTful Web Services — Roy Fielding \(a tese original, avançado\)](#)
- [HTTP Status Codes — referência rápida](#)
- [Postman Learning Center — testando APIs REST](#)

Docker

- [Docker — Get Started \(documentação oficial\)](#)
- [Play with Docker — ambiente Docker no browser. gratuito](#)

GitHub Actions

- [GitHub Actions — Quickstart](#)

Dicas para o Dia-a-Dia na Netline

Ferramentas que Usamos

- Editor de código: VS Code com extensões para TypeScript, ESLint e Prettier
- API Testing: Postman para testar endpoints e Swagger UI
- Base de Dados: DBeaver ou MySql WorkBench para inspecionar o PostgreSQL
- Gestão de Projetos: Linear
- Comunicação: Whatsapp

Princípios de Bom Código

DRY — Don't Repeat Yourself

Se escreves o mesmo código em dois sítios, extrai para uma função ou componente reutilizável. Repetição cria inconsistências e dificulta a manutenção.

KISS — Keep It Simple, Stupid

A solução mais simples que resolve o problema é quase sempre a melhor. Código complexo é difícil de entender, testar e manter.

Nomes Descritivos

Variáveis, funções e ficheiros devem ter nomes que descrevam claramente o que são ou fazem. Evite abreviações. Prefere `getUserById(id)` a `getU(i)`.

Commit Frequente & Mensagens Claras

Faz commits pequenos e frequentes com mensagens descritivas no imperativo: 'Adiciona validação de email no formulário' em vez de 'fix stuff'. Facilita revisões de código e encontrar bugs.

Próximos Passos Após Este Guia

Depois de completares as 6 fases, estarás preparado para contribuir em projetos reais. Os próximos temas a explorar incluem:

- Testes automatizados: Jest, Vitest, Testing Library
- Segurança web: OWASP Top 10, SQL Injection, XSS
- Monitorização e logs: estruturação de logs, métricas
- Cloud: AWS— serviços básicos
- Arquiteturas avançadas: microserviços, event-driven, WebSockets

Bem-vindo à equipa!

A aprendizagem é continua. Faz perguntas, experimenta, falha e aprende. Estamos aqui para ajudar.

Elaborado por: Israel Mateus Matusse (Analista Programador)